

TD - TP 7

Objectifs : programmation en langage C

- Pointeur de fonction
- Debugger
- gcc -S
- gcc -fverbose-asm

Références

- GNU debugger: <https://www.sourceware.org/gdb/>
- GNU data display debugger: <https://www.gnu.org/software/ddd/>
- commande `nm` (*name mangling*) :
`man nm`
[https://fr.wikipedia.org/wiki/Nm_\(Unix\)](https://fr.wikipedia.org/wiki/Nm_(Unix))

1. Reprendre le programme du TP2 question 3 (recherche d'une racine d'une fonction polynomiale par dichotomie). Exécuter ce programme sous le contrôle du debugger, soit en version ligne de commande par appel direct à `gdb`, soit en utilisant l'interface graphique `ddd`. On rappelle qu'il faut créer l'exécutable en spécifiant l'option `-g`. Exécuter le programme d'abord sans aucun point d'arrêt, puis avec.
Dans `ddd`, explorer les différents menus, en particulier **View** et **Program**. On pourra notamment activer **View -> Machine Code Window** pour observer le code assembleur correspondant au programme. Tester aussi l'action de **Data -> Print** et **Data -> Display**. Vous pourrez introduire un ou des bugs (un par un) et tester l'efficacité du debugger pour l'identification des bugs.
2. On s'intéresse à nouveau au programme `sw.c` du TP 1. Combien de caractères affiche ce programme ? Vérifier en utilisant un tube (pipeline) en s'inspirant de l'exemple du TP 0.
Dans l'instruction `switch`, un certain nombre d'entrées (`case`) sont inutiles car, au cours de l'exécution, le contrôle ne leur est jamais transmis ("on n'y passe jamais"). Identifier lesquelles. Vérifier à l'aide du debugger. Le confirmer aussi en écrivant une version épurée du programme (privé de ces entrées et des instructions inutiles) et qui donne bien sûr le même résultat.
3. On propose désormais d'explorer la commande `nm` du terminal. Pour cela, appliquer cette commande aux fichiers objets obtenus à partir des 3 fichiers `f?.c` présentés dans le TP2. Expliquer précisément l'information fournie à l'aide du manuel en ligne. Commenter et décommenter des instructions dans les différents fichiers, prévoir puis constater le résultat.
4. On donne le fichier `fic.o` sous l'espace Moodle. Identifier les symboles externes présents dans ce fichier. Combien y a-t-il de variables externes ? Sachant qu'elles sont toutes de type `float`, quelle est la valeur initiale de chacune d'elles ? Combien y a-t-il de fonctions ? Sachant qu'elles ont toutes le même prototype que les fonctions de la question précédente, que réalise chacune d'elles ? Des variables externes y sont-elles utilisées ?
5. A l'aide de l'option `-S` de `gcc`, créer les fichiers `add.s` et `sub.s` contenant le code en langage d'assemblage correspondant aux fichiers `add.c` et `sub.c` écrits lors du TP2. Comparer ces deux fichiers.
6. Dans quel registre est renvoyé le résultat des fonctions de la question précédente ? L'option `-fverbose-asm` est utile pour faire le lien avec les instructions du code source. Modifier légèrement la programmation, par exemple en ajoutant une instruction intermédiaire, et observer le code assembleur obtenu.